The Project :

I developed a numerical simulation software to solve the two-dimensional Brusselator reaction-diffusion problem, a system of coupled, nonlinear partial differential equations describing the interaction and diffusion of chemical species over time. Since obtaining analytical solutions for this system is generally not feasible, numerical methods are required to approximate its solution. The purpose of developing this software was to automate this computationally intensive process, which otherwise involves repeatedly discretising equations, assembling and solving systems of algebraic equations, and advancing the solution through successive time steps. The software implements different numerical integration approaches, including explicit Runge–Kutta and implicit Euler methods, and enables automated parameter variations to investigate the system's behaviour and compare the accuracy and computational performance of different numerical approaches.

The Organization :

The repository was structured to maintain modularity and separation of concerns. The source code was divided into individual C++ files handling the physical model (brusselator.cpp), numerical integration (integrator.cpp), data output (output.cpp) and program execution (solver.cpp), with corresponding header files organised separately in the include directory. Reference data and simulation results were stored in dedicated directories. Testing was performed at two levels: unit tests in C++ to verify individual components, including the Laplacian, reaction and Jacobian implementations, and component-level tests using Python's subprocess library to execute the solver and evaluate its behaviour. The latter covered input validation, time-step sensitivity for the implicit scheme and reaction-parameter variation for the Runge–Kutta scheme. Separate Bash scripts automated the parameter studies. Finally, a README.md was provided to document the repository structure, build and execution instructions, parameter variation scripts, and organisation of simulation results, making the software easier to use and reproduce.

Technological decisions :

C++17 was selected as the programming language standard, offering modern language features, computational performance and broad compiler compatibility without requiring the newer C++20 standard. Eigen was chosen for its efficient sparse matrix operations and linear system solvers, particularly for handling the large, predominantly zero-valued Jacobian matrix. VTK was selected to write simulation results into standardised scientific data files, which can be opened in ParaView, a scientific visualisation and analysis application used to inspect simulation outputs through graphical representations of the computed fields. This allowed the numerical results to be analysed visually without developing a separate visualisation tool. GoogleTest was used to implement unit tests for individual mathematical components. For build management, CMake was used to configure dependencies, compilation and linking, with Make handling the actual build process. This combination provided a reproducible build workflow, although it introduced additional configuration and dependency management. Architecturally, a modular design with separate components for the physical model, numerical integration, output handling and program execution was adopted to improve maintainability, testability and extensibility. While this approach required managing additional source files and interfaces, it made individual components easier to modify and test independently.